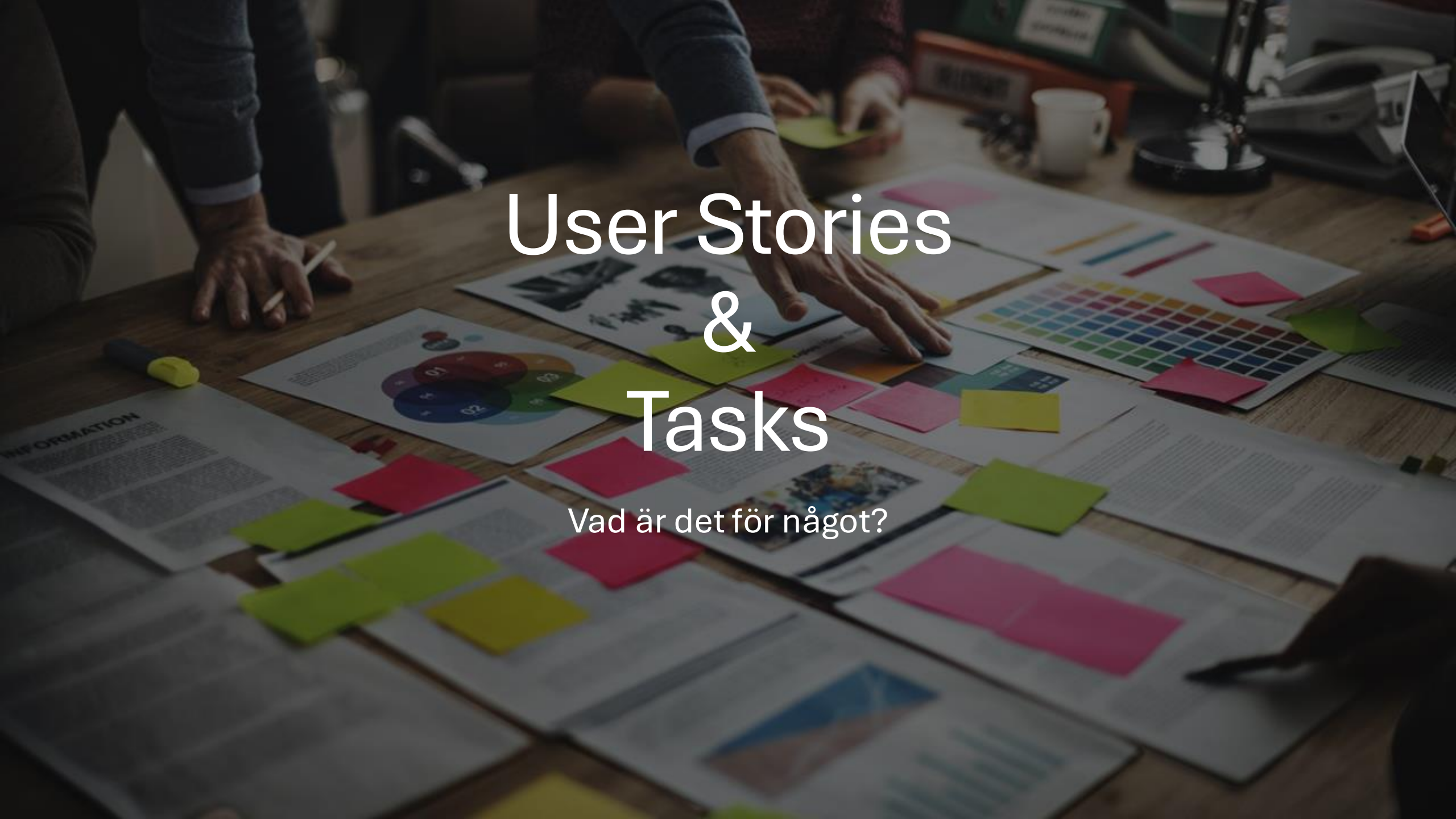


A photograph of a wooden table in a collaborative workspace. Several people's hands are visible, interacting with various documents and colorful sticky notes (pink, yellow, green). One document features a Venn diagram with three overlapping circles labeled 01, 02, and 03. Another document shows a color palette grid. The scene is dimly lit, with a desk lamp visible in the background. The text 'User Stories & Tasks' is overlaid in the center in a large, white, sans-serif font.

User Stories & Tasks

Vad är det för något?

Vad är User Stories?

En user story (US) är ett verktyg/process för att beskriva funktionalitet ur användarens perspektiv.

En US följer vanligtvis formatet:
"Som [roll] vill jag [funktion/behov] så att [värde/nytta]"

Varför User stories

I slutändan handlar det om att vi vill;

Ge en kund det den vill ha.
Producera innehåll.

Veta vad det är vi ska göra.



Genom att sätta en slutanvändare i fokus så är förhoppningen att vi skapar ett värde för denna.

forts. User stories

- US är utformade för att sätta en användare/roll och en viss funktion i fokus – utan att gå in på tekniska detaljer.
- Detta gör det “enklare” att (både för slutkund/användare och i utvecklingsteamet);
 - diskutera och komma fram till **funktionalitet** via i text beskrivna scenarion, eller stories, som utgår ifrån en **slutanvändare**.
 - **förståelse** av både tekniska och icke-tekniska personer.
 - göra det tydligt **varför** en viss funktionalitet behövs (dvs värdet för användaren).

INVEST

- INVEST-kriterierna – vad som utgör en bra US:
- **Independent** (Oberoende): Storyn kan implementeras separat från andra
- **Negotiable** (Förhandlingsbar): Detaljer kan diskuteras under utvecklingen
- **Valuable** (Värdefull): Den ger tydligt värde för användaren
- **Estimable** (Uppskattningsbar): Teamet kan uppskatta och estimerar tidsåtgång och/eller komplexitet
- **Small** (storlek): Tillräckligt liten för att genomföras i en sprint/iteration.
- **Testable** (Testbar): Det går att verifiera när den är klar



Ett exempel – “To-Do app”



Ta fram en enkel To-Do App

- *En app där användare kan skapa, organisera och hålla koll på sina uppgifter, sina To-Do's.*
- Ta fram User stories.

Exempel på US

Man ska kunna lägga till uppgifter.

Som *användare* vill jag kunna *skapa nya uppgifter* med titel och deadline så att jag kan hålla reda på vad jag behöver göra och när.

Jag vill kunna vill jag kunna markera uppgifter som slutförda med checkboxar.

Som *användare* vill jag kunna *markera uppgifter som slutförda* så att jag får en översikt över vad jag har gjort och vad som återstår.

Som användare vill jag kunna kategorisera mina uppgifter så att jag kan bli klar-

Som *användare* vill jag kunna *kategorisera mina uppgifter (arbete, privat, studier)* så att jag lättare kan fokusera på en typ av uppgifter i taget.

Tasks

- US behöver brytas ned i görbara uppgifter (tasks) för de personer (team) som ska ta fram produkten/appen (eller vad det nu kan vara).
- Om vi har US i en backlog så är det i sprint planning vi plockar relevanta US för att bryta ned dessa i tasks.
- **US:** Beskriver vad och varför ur användarens perspektiv
- **Tasks:** Beskriver hur från utvecklarteamets perspektiv. Här kan alltså finnas tekniska detaljer.
- **Varför bryta ned i tasks upp?** Ökar teknisk förståelse, estimeringsförmåga och förenklar teamets arbete.

Hur bryta ned US till tasks?

- **Analysera US:** Förstå vad som behöver byggas (under t ex Sprint planning).
- **Identifiera arbetsmoment:** Resonera (eller brainstorma) kring tekniska detaljer eller tekniska komponenter som krävs.
- **Bryt ned i Tasks:** Dela upp i US i mindre görbara tasks - varje task bör vara:
 - Tydligt definierad
 - Estimerbar (helst 1/2-2 arbetsdagar, 4 – 16 timmar)
 - Möjlig att genomföra/implementeras av en person.
- **Validera:** Säkerställ att alla tasks tillsammans uppfyller hela storyn.
- **Dokumentera:** Lägg till tasks i ert projektsystem (GH Projects, Trello etc.)

Tasks – bryta ned US

Skapa ny uppgift

Som *användare* vill jag kunna *skapa nya uppgifter* med titel och deadline så att jag kan hålla reda på vad jag behöver göra och när.

Felhantering (2h)

för när uppgifter inte kan sparas

Skapa UI för inmatningsformulär (2h)

med fält för titel och datumväljare för deadline.

Validera (3h)

inmatade uppgifter (t.ex. att titel inte är tom, deadline är ett giltigt datum)

Lägg till bekräftelse (1h)

för användaren när en uppgift har skapats

Designa och implementera listvy (4h)

som visar nyligen skapade uppgifter

Skapa routes för CRUD operationer (12h)

För nya uppgifter i API:et.

Tasks – tips & tricks

Undvik “för” **stora tasks** (bör inte ta mer än 1-2 dagar) - bryt ner stora tasks i flera mindre.

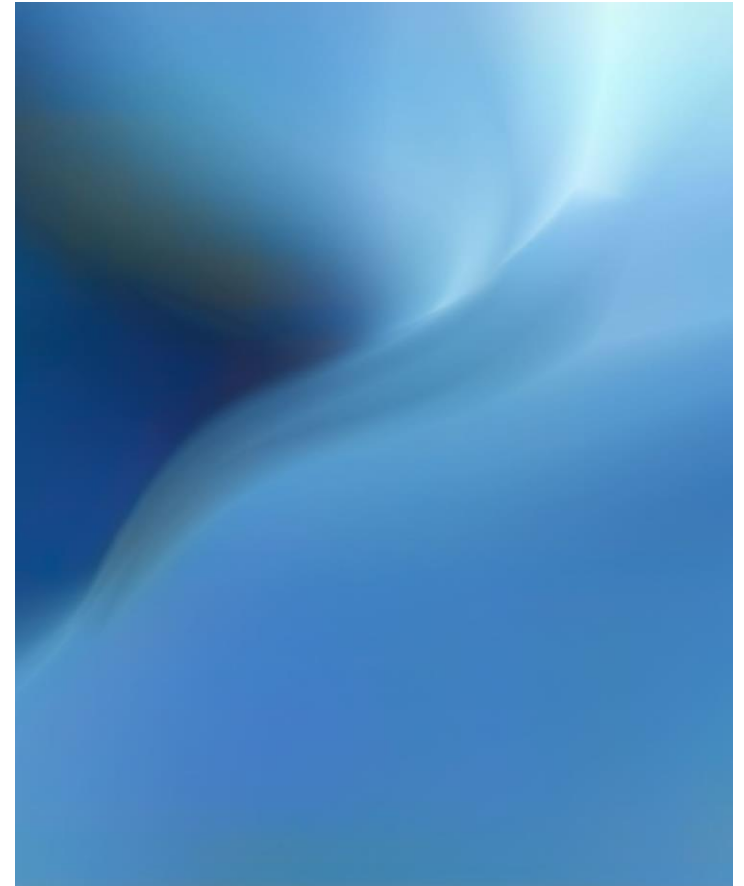
Glöm inte **dokumentation** – använd Trello, GH Projects etc. Assigna! Estimera!

Involvera hela teamet i nedbrytningen för att fånga alla och flera perspektiv.

Tänk på "Definition of Done" för varje task – **när är den färdig?** Acceptance criterias?

Inkludera alltid **testning** och **validering** – när är uppgiften fungerande och klar?

Var **konkret** men undvik “micromanagement” – (viss) tolkningsfrihet kring hur uppgiften ska lösas bör tillåtas.



Till sist – vad tar vi inte upp

- **User Story Mapping**

- Kan vara ett sätt att skapa User Stories på.
- För att organisera user stories och hur dessa hör ihop i ett flöde/serie.
- Kan vara bra för att inbördes prioritera arbetsordning samt visa förhållande/beroenden mellan olika US.

- **User Journey:**

Utifrån en användare och ett scenario kan man 'mappa' vilka US som blir involverade i vilken ordning.

- **Use Cases:**

Utifrån en användare och ett scenario, hur användaren interagerar med applikationen och dess funktionalitet.



Tack för att ni
lyssnade!